

2026 年网络与分布式系统课程实验题目

一、AI 算法

1. (1.0) 基于 RePB 框架的动态数据剪枝一致性修正机制研究。在深度学习训练中，动态数据剪枝 (Dynamic Data Pruning) 通过实时筛选“难样本”来加速训练。然而，现有方法 (如早期的动态剪枝策略) 面临两大核心挑战：一是得分漂移 (Score Context Drift)，即模型参数随时间变化，导致不同时刻计算的 Loss 得分无法直接比较；二是时间梯度偏差 (Temporal Gradient Bias)，即长期非均匀采样改变了数据的分布，导致优化方向偏离全量数据训练。

RePB (Resolving Pruning Biases) 提出了一种优雅的解决方案：局部窗口剪枝 (LWP) 和 累积时间重缩放 (CTR)。本实验要求同学们基于官方开源代码，在轻量级 CIFAR 数据集上复现 RePB 算法，通过控制变量法深入探究 LWP 如何解决得分漂移问题，以及 CTR 如何校准梯度方向，从而验证其在高剪枝率下的鲁棒性。

具体实验任务分以下三个小题。

(1) 基础题：环境搭建。能够成功运行官方代码，验证 RePB 在轻量级数据集上的可行性。RePB 的核心设计理念是“无侵入式”集成，即不需要修改模型结构，仅通过修改数据加载部分即可实现剪枝。你需要：

1. 搭建 Python 环境并安装 repb 库 (使用 `pip install git+https://github.com/mrazhou/RePB`)。
2. 找到官方脚本 `scripts/cifar.sh`，运行 CIFAR-10 上的 RePB 实验 (剪枝率设为 0.6，即只训练 40% 的数据)。
3. 探究：请在 PyTorch 的训练脚本中，指出 RePB 官方文档中提到的“修改三行代码”具体对应源码的哪三个位置？并解释这三行代码分别起到了什么作用。

(2) 提高题：机制剖析与“局部窗口”验证。深入理解 RePB 解决“Score Context Drift (得分漂移)”的核心创新：局部窗口剪枝 (LWP)，并分析超参数的影响。论文指出，因为模型参数的动态变化，直接比较不同时刻样本的 Loss 是不公平的，因此 RePB 提出了 Local Window Pruning (LWP) 来解决这个问题。你需要：

1. 查阅论文第 3.1 节 (Local Window Pruning)，理解参数 W (窗口大小) 的含义。
2. 在代码中找到控制窗口大小的逻辑 (通常与 Batch Size 相关)。设计实验：固定剪枝率，尝试模拟不同的窗口大小 (例如 $W=1$ 和 $W=\text{非常大}$ ，虽然代码可能默认固定，但学生需设计对比思路)。
3. 分析：根据论文图 2 (或实验部分的消融实验 Table 10)，如果我们将窗口大小 W 设置得非常大 (趋近于整个 Epoch)，RePB 的性能为什么会下降？这说明了什么问题？

(3) 进阶题：对比实验与“梯度对齐”思考。对比 RePB 与 InfoBatch 的差异，理解“累积时间重缩放” (CTR) 如何解决梯度偏差。InfoBatch 是 RePB 的主要对比方法。RePB 认为 InfoBatch 没有解决时间上的梯度偏差 (Temporal Gradient Bias)，且没有使用局部窗口。你需要：

1. 运行两个实验：
 - 实验 A: `bash scripts/cifar.sh RePB 0.6`
 - 实验 B: `bash scripts/cifar.sh InfoBatch 0.6` (需确认脚本是否支持相同剪枝率，若不支持则对比论文数据)
2. 观察：在高剪枝率 (60%) 下，对比两者的最终准确率。
3. 思考：根据论文第 3.3 节，RePB 使用了 Cumulative Temporal Rescaling (CTR) 来重加权 Loss。请解释为什么在动态剪枝中，我们需要根据样本的历史选择频率来调整 Loss 权重？如果去掉 CTR (即不做重缩放)，训练过程可能会出现什么现象 (例如某些类别性能变差)？

2. (1.1) 基于强化学习的大型语言模型推理能力增强与对齐优化实验。大型语言模型 (LLM) 的推理能力是当前人工智能领域的核心挑战。传统的监督微调 (SFT) 受限于人类标注数据的规模与质量, 难以覆盖复杂的逻辑推演过程。

参考文献《DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning》(arXiv:2501.12948v2), 本实验要求同学们探索通过强化学习 (RL) 激发模型推理潜能的机制。重点研究“思维链 (Chain-of-Thought)”作为隐式思维过程的涌现, 以及如何通过奖励模型 (Reward Model) 实现模型输出的对齐 (Alignment) 与语言一致性优化。具体实验任务分以下三个小题。

(1) 基础题: 对应论文章节: 1. Introduction (Chain-of-Thought) & 2. DeepSeek-R1-Zero。验证“思维链 (Chain-of-Thought, CoT)”是提升模型推理能力的关键, 复现论文中提到的“传统模型与推理模型的区别”。

实验步骤:

1. 选择模型: 在 Hugging Face 或 ModelScope 或 Ollama 上找到 Qwen2.5-1.5B (作为基座模型, 代表普通学生) 和 DeepSeek-R1-Distill-Qwen-1.5B (作为经过训练的模型, 代表学霸), 进行本地部署。
2. 设计考题: 学生自己编写 3 道“小学奥数”级别的文字题 (例如: “小明有若干苹果, 如果每人分 3 个会多 2 个, 分 4 个会少 1 个...”)。
3. 对比实验:
 1. 实验组 (Zero-Shot): 直接向两个模型提问, 看谁答对。
 2. 干预组 (Deep-Thinking): 向模型发出指令: “请先详细写出你的思考步骤 (Step-by-Step), 最后再给出答案。”
4. 成果验收:
 1. 列出编写的 3 道奥数题。
 2. 将上述模型部署到本地, 并成功运行, 截图。
 3. 统计两种模式下两个模型的正确率。
 4. 思考题: 为什么加上“Step-by-Step”后, 模型更容易答对? 这与论文中提到的“Supervised Fine-Tuning (SFT)”阶段有什么关系?

(2) 提高题: “自我反思”机制的模拟实现。对应论文章节: 2.3. Incentivize Reasoning Capability & 3.2. Training Details

实验目的: 模拟论文中提到的 GRPO (Group Relative Policy Optimization) 机制。虽然无法真的计算梯度, 但可以通过人工构建“多路径投票”来模拟 RL 的筛选过程。

实验步骤：

1. 多路径生成：针对同一道复杂的逻辑题，手动提示模型生成 4 次不同的回答（利用 Temperature 采样，或者提示“请给出四种不同的思路”）。
2. 人工“奖励模型”：你需要扮演论文中的 Reward Model。
 1. 阅读这 4 条回答。
 2. 制定一个简单的评分标准（例如：逻辑通顺 1 分，答案正确 1 分，格式规范 1 分）。
 3. 挑选出得分最高的那一条回答。
3. 模拟“进化”：将得分最高的那条回答作为范例，再次输入给模型，问：“参考下面这个完美的解题过程，请重新回答原问题。”
4. 撰写报告：
 1. 记录原始回答和“进化后”回答的区别。
 2. 思考题：在这个过程中，你扮演的“打分员”角色，对应论文中的哪个部分？如果把这个打分过程变成自动的程序，就是论文里的 RL 训练吗？

（3）进阶题：破解“幻觉”与“混合语”难题。对应论文章节：3. DeepSeek-R1 (Language Consistency Reward) & 6. Limitations。复现论文中提到的“语言一致性奖励（Language Consistency Reward）”和“有害内容控制（Safety）”，理解对齐（Alignment）的挑战。

实验步骤：

1. 复现 Bug：使用 DeepSeek-R1-Distill-Qwen-1.5B，输入一个混合了中英文的复杂问题，或者故意诱导模型说脏话/胡言乱语。

观察：模型是否会像论文提到的 R1-Zero 那样，出现中英混杂（Code-Switching）或者逻辑跳跃？

2. 设计“护栏”：学生尝试编写一个 System Prompt（系统提示词），作为“隐形的护栏”。
 1. 要求：强制模型只用中文回答，且必须先思考再回答，如果问题有害必须拒绝。

2. 示例：“你是一个严谨的 AI 助教。请用纯中文回答。在回答前，必须在心中用标签思考。如果问题涉及违法或不道德内容，请直接拒绝回答。”
3. 对抗测试：用刚才诱导失败的例子再次测试，看新的 System Prompt 是否有效。
4. 撰写报告：
 1. 展示修改前后的对话对比。
 2. 思考题：论文中提到 DeepSeek-R1 使用了额外的 Reward Model 来解决这些问题。你写的这个 Prompt，本质上是在模拟那个 Reward Model 的作用吗？

3. (1.2) 基于 UnCLe 框架的非线性时间序列动态因果发现实验。在复杂系统分析中，理解变量间的动态因果关系是核心挑战。传统的格兰杰因果检验往往假设关系是静态的，而现实世界（如人体运动、金融市场）中的因果关系往往是随时间演变的。UnCLe (UnCoupLing causality) 是一种基于深度学习的框架，它通过“解耦-再耦合”网络将时间序列转化为语义表示，并利用自回归依赖矩阵捕捉变量间的动态交互。然而，理解其如何通过扰动分析 (Perturbation Analysis) 精准定位动态因果，以及各组件（如依赖矩阵、参数共享机制）对因果发现的贡献，是掌握该方法的关键。参考论文《UnCLe: Towards Scalable Dynamic Causal Discovery in Non-linear Temporal Systems》(NeurIPS 2025) 的核心思想，本实验要求同学们在轻量级合成数据集上复现 UnCLe 模型，探究其在非线性动态系统中的因果发现机制。

具体实验任务分以下三个小题。

(1) 基础题：环境搭建与静态因果图复现。

理解 UnCLe 的基本原理，配置 Python 深度学习环境并克隆官方代码库。使用论文附录中提供的 NC8 数据集（非线性常量连接数据集，包含 8 个变量），该数据集具有复杂的非线性生成方程（如 $\sin$, cube , $\max$ 函数）。要求同学们跑通 UnCLe (A) 模式（即基于依赖矩阵 Ψ 聚合的静态因果发现模式）。任务包括：加载数据、训练模型（仅需少量 Epochs 以适应 CPU 环境）、从训练好的模型参数中提取静态因果图，并将其与 NC8 的真实因果图 (Ground Truth) 进行对比，计算准确率 (ACC) 和 AUROC 指标，验证模型在静态非线性场景下的基础重建能力。

(2) 提高题：动态因果发现与消融实验。

在基础题之上，进阶探究 UnCLe 的核心优势——动态因果发现。使用论文中的 TVSEM 数据集（时变结构方程模型，仅含 2 个变量但因果方向随时间翻转），要求实现 UnCLe (P) 模式（基于时间扰动的动态因果发现）。任务包括：实现时间序列的随机排列扰动 (Temporal Permutation) 逻辑，计算点对点的预测误差增益 (Error Gain)，并绘制因果强度随时间变化的热力图（类似论文 Figure 2）。要求观察模型是否能捕捉到因果关系在特定时间点（如 $t=400, 800$ ）的翻转。同时，进行消融实验：尝试移除论文中提到的“自回归依赖矩阵 (Auto-regressive Dependency Matrices)”，观察并分析模型性能 (AUPRC) 的下降情况，以此验证该组件在捕捉变量间交互作用中的必要性。

(3) 挑战题：扰动策略分析与参数敏感性研究。针对论文中提到的“扰动策略”进行深入对比分析。论文指出，时间重排 (Permutation) 优于置零 (Zero-Masking) 和加噪 (Noise Injection)。本题要求学生在 Lorenz#1（小规模洛伦兹系统）数据集上，复现论文 Table 4 的实验逻辑。任务包括：分别实现

Zero-Masking、Noise Injection 和 Temporal Permutation 三种不同的扰动策略。对比这三种策略在因果发现准确率（AUROC）上的差异，并结合理论分析：为什么仅仅“破坏时间顺序”比“破坏数值分布”（置零/加噪）更能有效地揭示因果关系？要求学生通过实验数据验证论文观点——有效的扰动必须在消除预测信息的同时保持变量的边际分布。

二、AI 基础设施

推荐使用 vLLM (<https://github.com/vllm-project/vllm>) 作为实验平台, 在真实 GPU 上完成实验可获得最真实的性能数据。没有 GPU 资源的同学可使用 llm-d-inference-sim 模拟器 (<https://github.com/llm-d/llm-d-inference-sim>) 或者自行设计算法原型模拟作为替代方案。

4. (1.1) LLM 推理服务的核心性能指标是首 Token 延迟 (TTFT)、逐 Token 延迟 (TPOT) 和端到端延迟 (E2E latency)。然而, 高吞吐量并不等同于高服务质量: 当请求无法在 SLO (Service Level Objective) 约束内完成时, 该请求的吞吐量对用户而言是无效的。DistServe 提出了 Goodput 的概念, 即满足 SLO 的有效吞吐量, 并指出 Prefill 与 Decode 阶段的相互干扰是导致 Goodput 下降的主要原因; Sarathi-Serve 进一步提出了 Chunked Prefill 和无阻塞调度策略来缓解该问题。本实验要求学生在单实例场景下深入理解这些核心指标的含义及其相互关系, 并设计前端控制策略来优化 Goodput。具体实验任务分为以下三个小题:

(1) 基础题: 搭建 1 个推理服务实例 (vLLM 或模拟器), 构造混合负载 (含短 prompt/长 prompt、短输出/长输出), 在不同并发度下采集 TTFT、TPOT、E2E latency、queue time 等指标, 绘制各指标随并发度变化的曲线。分析 Prefill 与 Decode 阶段的相互干扰现象, 判断负载饱和点。

(2) 提高题: 实现一个前端 SLO 感知的准入控制器。阅读并理解 DistServe 或 Sarathi-Serve 的核心思想, 在前端实现以下策略之一或组合: a) 基于请求长度的分队列调度, 短请求与长请求分别排队; b) 基于历史 TTFT/TPOT 的动态速率控制, 当 SLO 违反率升高时降低发送速率; c) 超过容量阈值时拒绝或延迟长请求。要求与无控制 baseline 对比 Goodput@SLO 指标, 能够分析所复现方法的优缺点有加分。

(3) 进阶题: 在提高题基础上, 设计一个长度感知与 SLO 感知的联合准入策略。系统需根据实时采集的指标在线自适应调整策略参数 (如队列阈值、速率上限), 目标是在隐藏测试 trace 上最大化评分: $\text{Score} = \text{Goodput@SLO} - \lambda \cdot \text{RejectRate}$ 。要求提交策略代码和实验报告, 报告需包含策略设计动机、算法描述、收敛性分析和消融实验。

参考文献:

[1] Zhong Y, Liu S, Chen J, et al. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. OSDI 2024.

[2] Agrawal A, Panwar A, Mohan J, et al. Sarathi-Serve: Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. OSDI 2024.

5.(1.2) 在 LLM 推理中, Prefill 阶段对 prompt 进行并行编码, 属于计算密集型; Decode 阶段逐 token 自回归生成, 属于访存密集型。当两个阶段共享同一实例时, 长 prompt 的 Prefill 会阻塞正在进行的 Decode, 导致 TPOT 抖动。DistServe 提出将 Prefill 和 Decode 解耦到不同的实例上 (P/D disaggregation), 通过 KV Cache 迁移实现两阶段的独立扩展。本实验要求学生对比单体部署与 P/D 解耦部署的性能差异, 并实现智能路由策略。具体实验任务分为以下三个小题:

(1) 基础题: 分别搭建两种部署模式——单体部署 (1 个实例处理完整请求) 和 P/D 解耦部署 (1 个 Prefill 实例 + 1 个 Decode 实例, 分别监听不同端口), 在相同混合负载下测量 TTFT、TPOT、E2E latency 和 queue time。要求观察并分析: 当长 prompt 到来时, 单体部署中 Decode 请求的延迟是否显著上升; P/D 解耦部署是否缓解了该问题; 两种模式各自的瓶颈在哪里。

(2) 提高题: 实现一个前端请求拆分与路由器。阅读并理解 DistServe 的 P/D 解耦思想或 Sarathi-Serve 的 Chunked Prefill 思想, 实现以下策略之一: a) 基于 prompt 长度的路由: 长 prompt 请求进入 P/D 解耦路径, 短 prompt 请求走单体路径; b) 简化版 Chunked Prefill: 将超长 prompt 切分为若干段, 顺序送入 Prefill 实例以减少单次 Prefill 的阻塞时间。要求与纯单体和纯 P/D baseline 对比性能, 能够分析方法优缺点有加分。

(3) 进阶题: 实现一个在线自适应 P/D 路由器, 能够根据实时负载动态决定: 哪些请求走单体路径、哪些走 P/D 路径、以及 P 实例与 D 实例的逻辑配比。评分指标: $\text{Score} = \text{Goodput@SLO} - \alpha \cdot \text{TransferOverhead}$ 。要求提交策略代码和完整实验报告, 报告需包含路由决策算法设计、自适应机制说明、以及不同负载模式下的鲁棒性分析。

参考文献:

- [1] Zhong Y, Liu S, Chen J, et al. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. OSDI 2024.
- [2] Agrawal A, Panwar A, Mohan J, et al. Sarathi-Serve: Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. OSDI 2024.

6. (1.0) KV Cache 是 LLM 推理中的关键资源，其容量直接限制了系统的并发处理能力。当多个请求共享相同的前缀（如系统提示词、few-shot 示例等）时，Prefix Caching 技术可以避免重复计算，显著降低 TTFT。Mooncake 提出了以 KV Cache 为中心的分布式推理架构，通过全局缓存感知调度将相同前缀的请求路由到已缓存该前缀的实例。本实验的核心不是实现底层缓存机制，而是设计前端路由策略来最大化前缀局部性。实验任务分为以下三个小题：

（1）基础题：搭建 2-4 个推理服务实例，实现一个最简单的 Round-Robin 路由器，向所有实例发送混合负载。构造两类请求 trace：一类具有大量共享前缀（如多个请求共享同一系统提示词），另一类为完全随机前缀。在 Round-Robin 路由下测量各实例的 TTFT、queue time 和负载分布。分析：在 Round-Robin 策略下，共享前缀的请求是否被分散到不同实例，前缀局部性是否被破坏。

（2）提高题：实现一个前缀亲和性路由器。阅读并理解 Mooncake 的缓存感知调度思想，在路由器中维护前缀到实例的映射表，将相同前缀哈希的请求优先路由到同一实例以提升缓存命中率。以 Round-Robin 路由作为 baseline，对比 TTFT、queue time 和实例间负载均衡度。使用真实 vLLM 的同学可额外对比 prefix cache hit ratio。能够分析 Mooncake 方法优缺点有加分。

（3）进阶题：实现一个全局缓存感知路由器，解决提高题中前缀亲和性路由可能导致的热点问题。支持以下功能：a) 热门前缀可复制到多个实例以分散负载；b) 冷门前缀仅保留单副本以节省资源；c) 前缀热度发生漂移时动态迁移前缀归属。评分指标： $\text{Score} = -\text{AvgTTFT} - \beta \cdot \text{LoadImbalance}$ 。要求提交完整策略代码和实验报告，报告需包含缓存放置策略设计、热度检测与迁移机制、以及多种负载模式下的对比实验。

参考文献：

- [1] Qin R, Li Z, He W, et al. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. FAST 2025.
- [2] Yao J, Li H, Liu R, et al. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. EuroSys 2025.

7.(1.1)在多实例 LLM 推理集群中，传统的负载均衡策略（如 Round-Robin、Least-Connections）未考虑请求长度的异质性。由于不同长度的请求消耗的计算和缓存资源差异巨大，简单的均衡策略往往导致严重的尾延迟。L4 提出了长度感知调度，将请求按 prompt 长度分桶并路由到不同的实例组，从而在每个实例组内实现更均匀的资源消耗。本实验要求学生实现并对比多种前端负载均衡策略。具体实验任务分为以下三个小题：

（1）基础题：启动 2-4 个推理服务实例作为服务节点，实现 Round-Robin 和 Least-Queue 两种前端路由策略。构造包含短、中、长三类 prompt 的混合负载，测量 p50/p95/p99 E2E latency、p95 TTFT、吞吐量和实例间负载不均衡度。要求分析两种策略在不同负载强度下的表现差异。

（2）提高题：实现一个长度感知路由。阅读并理解 L4 的核心调度思想，按 prompt 长度将请求分桶（如短请求池、中请求池、长请求池），不同桶路由到不同实例组。与 Least-Queue baseline 对比，重点分析尾延迟（p95/p99）的改善情况。能够分析 L4 方法的优缺点有加分。

（3）进阶题：实现一个在线分桶边界自适应算法。系统运行过程中，根据最近时间窗口的 TTFT、queue time 和各实例利用率，动态调整“短/中/长”的长度边界，使系统在负载分布变化时仍能保持良好的尾延迟。评分指标： $Score = -p95_E2E - \delta \cdot LoadImbalance$ 。要求提交策略代码和实验报告，报告需包含自适应算法的设计与收敛分析、边界迁移策略说明、以及在多种 trace 上的鲁棒性实验。

参考文献：

[1] Zheng Z, Huang S, Chen Q, et al. L4: Low-Latency and Load-Balanced LLM Serving via Length-Aware Scheduling. Preprint, arXiv:2512.19179, 2024.

[2] Kwon W, Li Z, Zhuang S, et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023.

8. (1.2)随着大模型定制化需求的增长, LoRA (Low-Rank Adaptation) 成为主流的轻量级微调方法。在多租户推理场景中, 不同用户可能绑定不同的 LoRA 适配器, 如何高效管理和调度这些适配器成为关键挑战。S-LoRA 提出了统一的多 LoRA Serving 框架, 通过自定义 CUDA 内核和适配器内存管理实现高效的多租户服务; CaraServe 进一步优化了适配器的预热和切换策略。在真实 vLLM 上, 可通过 S-LoRA 功能加载多个 LoRA 适配器并观测实际的 adapter 切换开销。具体实验任务分为以下三个小题:

(1) 基础题: 构造多租户请求 trace, 不同请求绑定不同的 LoRA adapter ID, 发送到 1-2 个推理服务实例。采集并统计: 每个 adapter 的请求量分布、各租户的 TTFT 和 E2E tail latency、以及不同租户之间的服务公平性。使用真实 vLLM 的同学还需统计 adapter 切换频率及其对延迟的影响。

(2) 提高题: 搭建 2-4 个推理服务实例, 实现一个适配器感知路由器。阅读并理解 S-LoRA 或 CaraServe 的核心调度思想, 将同一 adapter 的请求优先路由到同一实例, 热门 adapter 可绑定固定实例池。以随机分配作为 baseline, 对比 tail latency、queue time、每实例 adapter 多样性和服务公平性。能够分析论文方法的优缺点有加分。

(3) 进阶题: 实现一个热度感知的适配器放置与预取策。系统需支持: a) 热门 adapter 提前预热到多个实例, 降低冷启动概率; b) 长尾 adapter 按需动态加载; c) 热度漂移时触发适配器重新放置。评分指标: $\text{Score} = -p95_latency - \eta \cdot \text{Unfairness} - \mu \cdot \text{AdapterDiversity}$ (每实例需同时维护的 adapter 种类数)。要求提交策略代码和实验报告, 报告需包含热度建模方法、放置算法设计、预取触发机制、以及在不同热度分布下的对比实验。

参考文献:

[1] Sheng Y, Cao S, Li D, et al. S-LoRA: Serving Thousands of Concurrent LoRA Adapters. MLSys 2024.

[2] Li S, Lu H, Wu T, et al. CaraServe: CPU-Assisted and Rank-Aware LoRA Serving for Generative LLM Inference. Preprint, arXiv:2401.11240, 2024.

三、AI 数据存储

9. (1.0) 基于 Learned Index 的可更新索引设计与优化实验。随着数据规模的不断增长，传统索引结构在查询效率和空间开销之间面临越来越大的权衡问题。Learned Index 通过使用机器学习模型拟合键值分布，将索引过程转化为位置预测问题，为数据库索引优化提供了新的思路。然而，在支持动态插入更新时，Learned Index 往往会面临插入性能下降、空间利用率降低以及结构退化等问题。参考论文《High Performance or Low Memory? An Updatable Learned Index Framework for Time-Space Tradeoff》(SIGMOD2025) 的相关思想，本实验要求同学们设计并实现一个支持动态更新的 Learned Index 系统，理解预测误差、节点密度与索引性能之间的关系，并探索时间与空间之间的平衡。

具体实验任务分以下三个小题。

- (1) 基础题：理解 Learned Index 的基本原理，实现简化版 Learned Index，完成有序键值数据的存储、线性模型位置预测以及基于误差范围的 last mile search，实现基本的查询功能，并测试查询延迟和预测误差等指标。
- (2) 提高题：阅读并复现论文，要求在基础索引结构上支持插入操作，并通过预留 gap 的方式降低插入时的数据移动开销。要求实现插入、查询和范围扫描功能，并对比无 gap 策略和有 gap 策略在查询性能、插入性能和空间利用率方面的差异。
- (3) 进阶题：在前述基础上进一步提出并实现至少一种优化方法，例如根据插入频率自适应调整 gap 分布、实现节点分裂与重构机制、分析重复键或密集插入场景下的索引退化问题等，并通过实验验证优化效果。

10.(1.1) 基于图索引的高维近似最近邻搜索优化实验。高维近似最近邻(ANN)搜索是向量数据库、推荐系统和信息检索中的关键技术。图索引方法(如 HNSW)因检索效率高、效果好而被广泛应用,但在高维场景下,查询过程中频繁的距离计算会带来较大的性能开销。参考论文《Accelerating High-Dimensional ANN Search via Skipping Redundant Distance Computations》(SIGMOD2025)的相关思想,本实验要求同学们围绕图索引 ANN 搜索的性能优化展开设计与实现,分析系统瓶颈,并探索减少冗余距离计算的方法。

具体实验任务分以下三个小题。

- (1) 基础题: 要求使用开源 ANN 检索工具(如 hnswlib 或 Faiss)完成基础图索引检索系统的搭建,实现向量数据导入、索引构建和 top-k 最近邻查询,并测试 recall@10、查询延迟和吞吐量等基本性能指标。
- (2) 提高题: 阅读并复现论文,要求在基础图索引检索系统上实现简化版候选筛选优化机制,例如利用随机投影或 PCA 将高维向量映射到低维空间,在图搜索过程中先利用低维距离筛选候选点,再对更有可能进入结果集的点进行精确距离计算,以减少冗余距离计算。
- (3) 进阶题: 在前述基础上进一步实现至少一种优化方法,例如部分距离提前终止、不同低维映射方法比较、缓存优化或多线程并行搜索等,并通过实验验证优化效果。

11. (1.1) 面向分布不均衡数据的近似最近邻图索引优化实验。在推荐系统、搜索引擎和智能问答等应用中,大规模向量检索系统需要在保证准确性的同时尽可能提高查询效率。图索引方法(如 HNSW)已成为当前近似最近邻(ANN)搜索的重要技术方案。然而,真实数据中的向量分布往往存在明显的不均衡特征:部分向量聚集成簇,容易被快速检索,而部分向量分布稀疏,难以通过图搜索有效访问。这种分布差异会导致高召回场景下的搜索成本显著增加。参考论文《Vista: Vector Indexing and Search for Large-scale Imbalanced Datasets》的相关思想,本实验要求同学们围绕分布不均衡场景下的图索引检索展开设计与实现,分析数据分布特征对检索性能的影响,并探索针对难检索点的优化方法。

具体实验任务分以下三个小题。

- (1) 基础题:使用开源 ANN 检索工具(如 hnswlib 或 Faiss)完成基础图索引检索系统的搭建,实现向量数据导入、索引构建和 top-k 最近邻查询,并测试 recall@10、查询延迟和吞吐量等基本性能指标,熟悉图索引 ANN 检索的基本原理。
- (2) 提高题:阅读并复现论文,要求在基础图索引检索系统上实现简化版分布感知优化机制,例如通过统计向量的近邻密度、近似入度或局部聚类特征,识别难检索点,并为这些点分配更大的候选搜索预算或补充额外连接边,以提升其可达性。要求对比优化前后的 recall 和查询性能,分析不同数据分布条件下的效果。
- (3) 进阶题:在前述基础上进一步提出并实现至少一种优化方法,例如设计更合理的难点判定指标、比较不同补边策略、探索局部聚类感知的建图方法,或结合课程特点实现多线程并行搜索优化,并通过实验验证优化效果。

12. (1.2) 支持属性过滤的近似最近邻向量检索系统设计与优化。随着向量数据库、推荐系统和检索增强生成 (RAG) 等应用的发展, 高维向量相似性检索已成为数据库与分布式系统中的重要问题。在实际应用中, 用户往往不仅需要检索与查询向量相似的数据, 还希望满足额外的属性条件, 例如商品类别、价格区间等。传统近似最近邻 (ANN) 检索方法 (如 HNSW) 在纯向量检索场景中效率较高, 但难以直接支持属性过滤查询。参考论文《Beyond Vector Search: Querying With and Without Predicates》(SIGMOD2025) 的相关思想, 本实验要求同学们设计并实现一个支持属性过滤的 ANN 检索系统, 并在此基础上进行性能优化。

具体实验任务分以下三个小题。

- (1) **基础题:** 使用开源 ANN 检索工具 (如 `hnswlib` 或 `Faiss`) 完成基础向量检索系统的搭建, 实现向量数据导入、索引构建和 `top-k` 最近邻查询, 并测试 `recall@10`、查询延迟和索引构建时间等基本性能指标。
- (2) **提高题:** 阅读并复现论文, 要求在基础 ANN 检索系统上增加属性过滤功能, 支持类别属性和数值范围属性的条件查询。可采用预过滤、后过滤或倒排索引等方法, 并与无过滤 ANN 和 `brute-force` 方法进行性能对比, 分析不同属性选择率下的查询效果和性能变化。
- (3) **进阶题:** 在前述基础上提出并实现至少一种优化方案, 例如设计属性感知跳转边、自适应过滤策略、缓存优化或多线程并行查询等, 并通过实验验证优化效果。